

Lecture Introduction

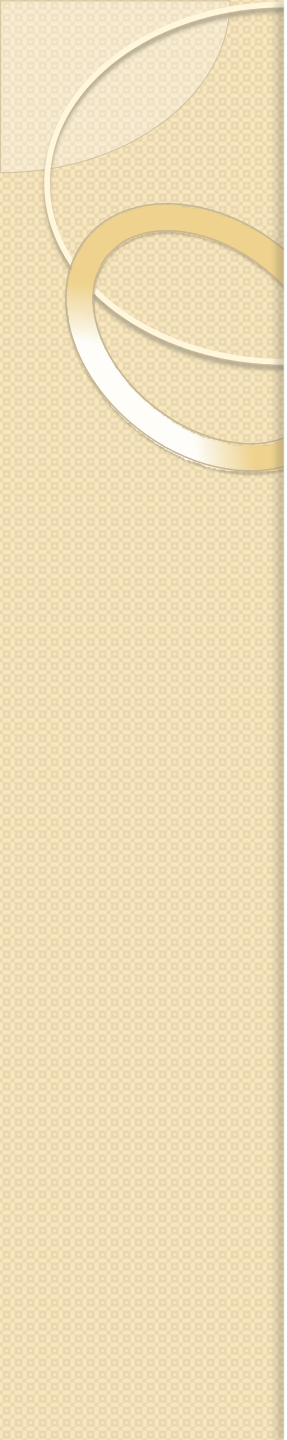
1. Anomalies Intro and It's Types
2. Anomalies Examples
3. Database Normalization and it's types
4. Practice Examples
5. Identification of Normalization Types



Database Anomalities

What is an Anomaly?

- Definition
 - Problems that can occur in poorly planned, un-normalized databases where all the data is stored in one table (a flat-file database).



Insert Anomaly

Insert Anomaly

- An **Insert Anomaly** occurs when certain attributes cannot be inserted into the database without the presence of other attributes.

Insert Anomaly

Let's take a look at the following table:

Customer	Purchase date	Product name	Amount	Price	Total price
Joe Smith	2014-02-14	Yoga mat	1	80	80
Jane Bauer	2014-02-16	Yoga block	2	30	60
Joe Smith	2014-02-14	Yoga block	2	30	60
Joe Smith	2014-02-14	Yoga strap	1	10	10
Thomas Apple	2014-02-18	Dumbbells 2kg	2	30	60
Jane Bauer	2014-02-16	Yoga mat	1	80	80

Insert anomalies

It's impossible to insert a product into the table if the product hasn't been bought by a customer yet. Similarly, it's impossible to insert a customer who hasn't made a purchase yet.

Delete Anomaly

Delete Anomaly

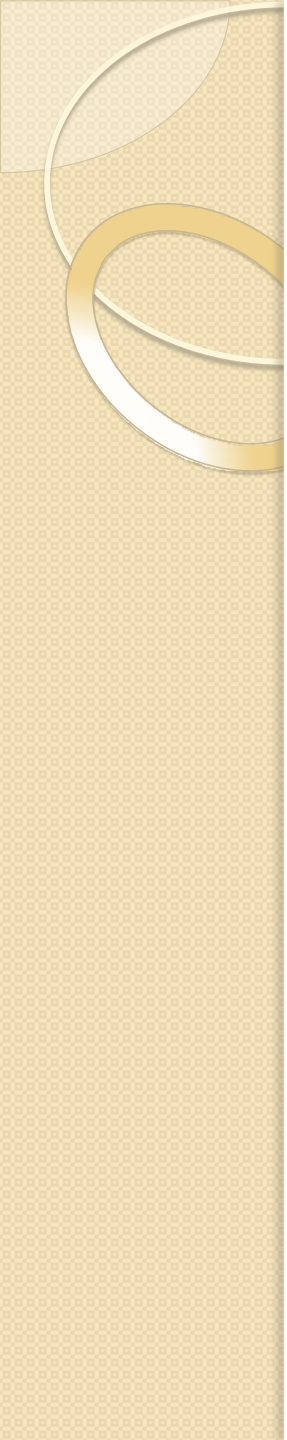
- A **Delete Anomaly** exists when certain attributes are lost because of the deletion of other attributes.

Delete Anomaly

Customer	Purchase date	Product name	Amount	Price	Total price
Joe Smith	2014-02-14	Yoga mat	1	80	80
Jane Bauer	2014-02-16	Yoga block	2	30	60
Joe Smith	2014-02-14	Yoga block	2	30	60
Joe Smith	2014-02-14	Yoga strap	1	10	10
Thomas Apple	2014-02-18	Dumbbells 2kg	2	30	60
Jane Bauer	2014-02-16	Yoga mat	1	80	80

Delete anomalies

If you delete the Thomas Apple purchase (say, because the order was cancelled), you will also delete the product "Dumbbells 2kg."



Update Anomaly

Update anomalies

It's difficult to update data in the table. If you want to change the name of the product, you have to update all rows where the product is bought. You cannot change the price of the product for all future purchases.

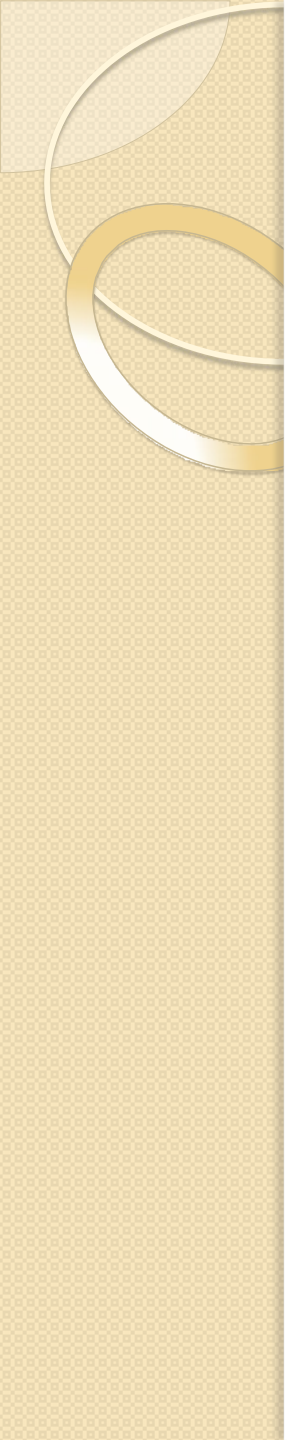
Data Normalization

What is Normalization?

Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies. Normalization rules divides larger tables into smaller tables and links them using relationships



Customer	Purchase date	Product name	Amount	Price	Total price
Joe Smith	2014-02-14	Yoga mat	1	80	80
Jane Bauer	2014-02-16	Yoga block	1	30	60
Joe Smith	2014-02-14	Yoga block	1	30	60
Joe Smith	2014-02-14	Yoga strap	1	10	10
Thomas Apple	2014-02-16	Dumbbells: 2kg	1	30	60
		Yoga mat	1	80	80



First Normalization

1NF (First Normal Form) Rules

- Each table cell should contain a single value.
- Each record needs to be unique.

There should be no composite or Multi-valued attributes in 1st Normalization Form

First Normalization

Example

item	colors	price	tax
T-shirt	red, blue	12.00	0.60
polo	red, yellow	12.00	0.60
T-shirt	red, blue	12.00	0.60
sweatshirt	blue, black	25.00	1.25



Example

item	color	price	tax
T-shirt	red	12.00	0.60
T-shirt	blue	12.00	0.60
polo	red	12.00	0.60
polo	yellow	12.00	0.60
sweatshirt	blue	25.00	1.25
sweatshirt	black	25.00	1.25

First Normalization

Id	First,Last
1	A,B



Id	First	Last
1	A	B

Second Normalization

1. A table that is in 1NF .
2. A table is in 2NF if each non-key (non primary and/or candidate keys) column depends on ALL candidate key.
3. There should be no **Partial Functional Dependency**.
4. **Non-Attribute should not be dependent on Part of Primary Key.**

Second Normalization

Example of a table not in 2NF:

<u>CourseID</u>	<u>SemesterID</u>	Num Student	Course Name
IT101	201301	25	Database
IT101	201302	25	Database
IT102	201301	30	Web Prog
IT102	201302	35	Web Prog
IT103	201401	20	Networking

Primary Key

The *Course Name* depends on only *CourseID*, a part of the primary key not the whole primary {*CourseID*, *SemesterID*}. It's called **partial dependency**.

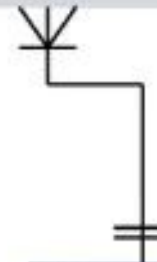
Solution:

Remove *CourseID* and *Course Name* together to create a new table.

<u>CourseID</u>	<u>Course Name</u>
IT101	Database
IT101	Database
IT102	Web Prog
IT102	Web Prog
IT103	Networking

Done? Oh no, it is still not in 1NF yet. Remove the repeating groups too. Finally, connect the relationship.

<u>CourseID</u>	<u>SemesterID</u>	Num Student
IT101	201301	25
IT101	201302	25
IT102	201301	30
IT102	201302	35
IT103	201401	20



<u>CourseID</u>	<u>Course Name</u>
IT101	Database
IT102	Web Prog
IT103	Networking

Third Normalization

1. A table that is in 1NF and 2NF keys.)
2. For 2NF we should remove partial dependency and for 3NF we should remove **transitive dependency**
3. **Non-Prime attributes should not Manage/Depends Non-Prime attributes**
4. **A relation schema R is in third normal form (3NF) if, whenever a nontrivial functional dependency $X \rightarrow A$ holds in R, either (a) X is a Super of R, or (b) A is a prime attribute of R**

Prime attributes are those attributes that are part of the candidate key . Non-Prime attributes are those attributes that are not part of the candidate key

Third Normalization

Student_Detail

Stu_ID	Stu_Name	City	Zip
--------	----------	------	-----



We find that in the above Student_detail relation, Stu_ID is the key and only prime key attribute. We find that City can be identified by Stu_ID as well as Zip itself. Neither Zip is a superkey nor is City a prime attribute. Additionally, $\text{Stu_ID} \rightarrow \text{Zip} \rightarrow \text{City}$, so there exists **transitive dependency**.

To bring this relation into third normal form, we break the relation into two relations as follows –

Student_Detail

Stu_ID	Stu_Name	Zip
--------	----------	-----

ZipCodes

Zip	City
-----	------

BCNF Normalization

1. A table that is in 3NF.)
2. **Non-Prime attributes should not Manage Prime attributes.**
3. **For BCNF, the table should be in 3NF, and for every FD, LHS is Prime attribute.**
4. **A relation schema R is in BCNF if whenever a nontrivial functional dependency $X \rightarrow A$ holds in R, then X is a Super key of R.**

BCNF Normalization

student_id	subject	professor
101	Java	P.Java
101	C++	P.Cpp
102	Java	P.Java2
103	C#	P.Chash
104	Java	P.Java

Functional Dependencies =:

{student_id,subject}->professor

{professor}->subject (Here non prime professor is determining part of candidate key (subject). It is violating BCNF)

Note:

Prime attributes are combination (student_id,subject)

BCNF Normalization

Note:

Q There is a question , why did we not combine student_id with Professor?

Because it will result in partial dependency between Professor ->subject. Because we are solving BCNF That is why 2nd and 3rd Normal Forms should be achieved. So , we have left with one choice to create composite Prime attribute (student and subject).

BCNF Normalization(Solution)

Student_id	professor
101	P.Java
101	P.Cpp
102	P.Java2
103	P.Chash
104	P.Java

professor	subject
P.Java	Java
P.Cpp	C++
P.Java2	Java
P.Chash	C#

How to find the highest normal form of a relation in DBMS?

Steps to follow to find the highest normal form of a relation

1. The first step is to find all feasible candidate keys of the relation and its attributes.
2. The second step is to organize into two categories all the attributes of the relation:
 1. Prime attributes
 2. Non-prime attributes
3. Third and the last step is to examine to determine for 1st normal form and then 2nd and so on. If the process is unsuccessful in satisfying nth normal form condition, then the highest normal form will be n-1.

How to find the highest normal form of a relation in DBMS?

Problem 1) Find the highest normal form of a relation $R(A,B,C,D,E,F)$ with Functional dependency set as $AB \rightarrow CDE, D \rightarrow BE$.

Prime ATTRIBUTES or CK = {AB,AD}

BCNF: AB satisfies (non –prime attributes not determining prime attributes) , But $D \rightarrow BE$, LHS is not Super key , so it violates BCNF.

3rd Nf: AB satisfies, but $D \rightarrow BE$, LHS is not super key and in RHS B is not prime.

2nd NF: In the case of AB (there is no partial dependency) , But in the case of $D \rightarrow BE$, non-prime attributes are dependent on part of candidate key (D). So , it is not 2nd NF also.

The above relation is in Only 1st NF.

How to find the highest normal form of a relation in DBMS?

Problem 2) Find the highest normal form of a relation $R(P, Q, R, S, T)$ with Functional dependency set as $(QR \rightarrow S, PR \rightarrow QT, Q \rightarrow T)$.

How to find the highest normal form of a relation in DBMS?

Step 1:

As the relation $(PR)^+ = (P, Q, R, S, T)$ is given, but not a single of its subset can determine all attributes of relation, So PR will be candidate key. P or R can't be derived from any other attribute of the relation, so there will be only one candidate key (PR).

Step 2:

- a. The attributes which are part of candidate key (P, R) are Prime attributes.
- b. The others will be non-prime attributes (Q, S).

Step 3:

A Relational Database Management System does not enable multi-valued or composite attribute. So, the relation $R(P, Q, R, S, T)$ is in 1st normal form.

Because $QR \rightarrow S$ is in 2nd normal form (QR is not a proper subset of candidate key PR) and $PR \rightarrow QT$ is in 2nd normal form (PR is candidate key) and $Q \rightarrow T$ is in 2nd normal form (Q is not a proper subset of candidate key PR). So, the relation is in 2nd normal form.

Because in $QR \rightarrow S$ (neither QR is a super key nor S is a prime attribute) and in $Q \rightarrow T$ (neither Q is a super key nor T is a prime attribute) but to satisfy 3rd normal form, either LHS of a Functional Dependency should be super key or RHS should be prime attribute. So, the relation is not in 3rd normal form.

So, the highest normal form of relation will be 2nd Normal form.